

Partitioning for Parallel Loops/Invoke

- Split according to available worker threads

Range Partitioner



with indexing
(Parallel.For)

Chunk Partitioner



with iteration
(Parallel.ForEach)

GUI Thread Model

- Only single-threading
 - Only a special UI-thread is allowed to access UI-components
- UI thread
 - Loop to process the event queue

why only one UI thread?

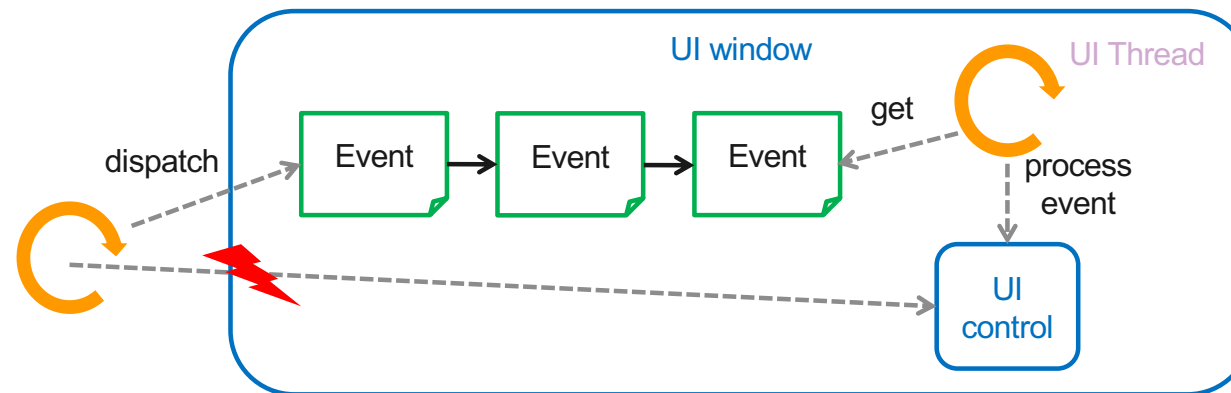
- is more needed? user can only do something at a time
- concurrency issues with more UI threads
- thread confinement: only UI Thread can change UI control

only UI thread can update UI

button press is an event

in app: click button to show prime numbers, foreign thread (non ui thread) calculates prime numbers (is a long operation), now the foreign threads creates event to show the result for UI thread (launch task with dispatch)

important: event in event queue for UI Thread should not take long.



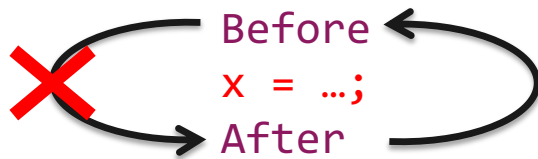
[] anki karte machen

.NET Volatile: Half Fence

- **Volatile Write: Release semantics**
 - Preceding memory accesses are not moved below it
- **Volatile Read: Acquire semantics**
 - Subsequent memory accesses are not moved above it

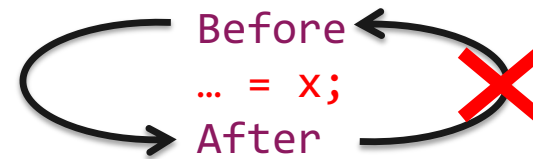
volatile int x;

Volatile Write



Allowed reordering

Volatile Read



Write: alles bis zu der volatile Variable wird garantiert ausgeführt, aber in C# darf auch schon etwas von danach ausgeführt worden sein.

Read: alles nach volatile Var wird auch erst nachher gemacht, aber es darf auch Sachen von davor erst danach machen

Operational intensity

- More calculations relative to the number of memory accesses -> compute bound
- Larger time spent on memory accesses relative to the calculations -> memory bound
- Measure: a ratio called **Operational intensity**.
- *Operational intensity is most often defined as a FLOPs per Byte ratio*
- Operational Intensity = $\frac{\text{operations per second}}{\text{bytes per second}} = \frac{\text{FLOPs}}{\text{Bytes}}$
- Higher the OI the better -> efficient utilization of modern parallel processors

- *Compute Bound*

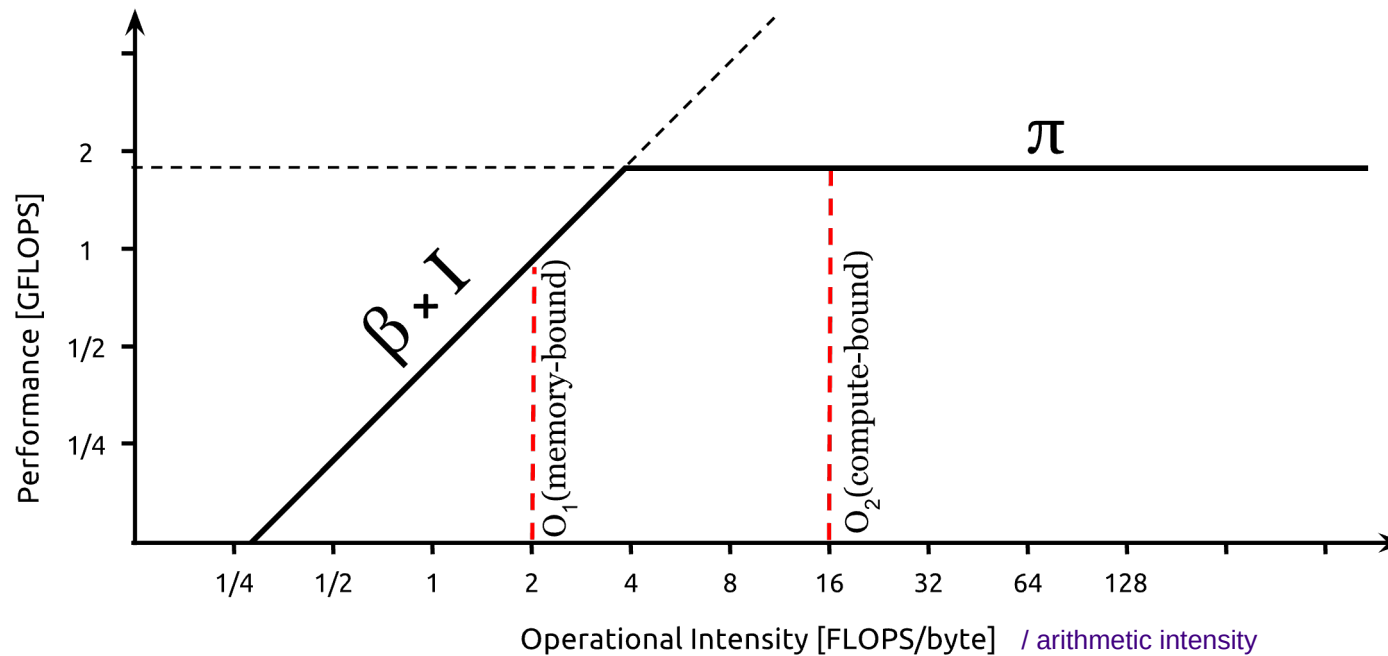
- $t_{\text{compute}} > t_{\text{memory}}$
- $\frac{\#ops}{BW_{\text{compute}}} > \frac{\#bytes}{BW_{\text{memory}}}$
- $\frac{\#ops}{\#bytes} > \frac{BW_{\text{compute}}}{BW_{\text{memory}}}$

- *Memory Bound*

- $t_{\text{compute}} < t_{\text{memory}}$
- $\frac{\#ops}{BW_{\text{compute}}} < \frac{\#bytes}{BW_{\text{memory}}}$
- $\frac{\#ops}{\#bytes} < \frac{BW_{\text{compute}}}{BW_{\text{memory}}}$

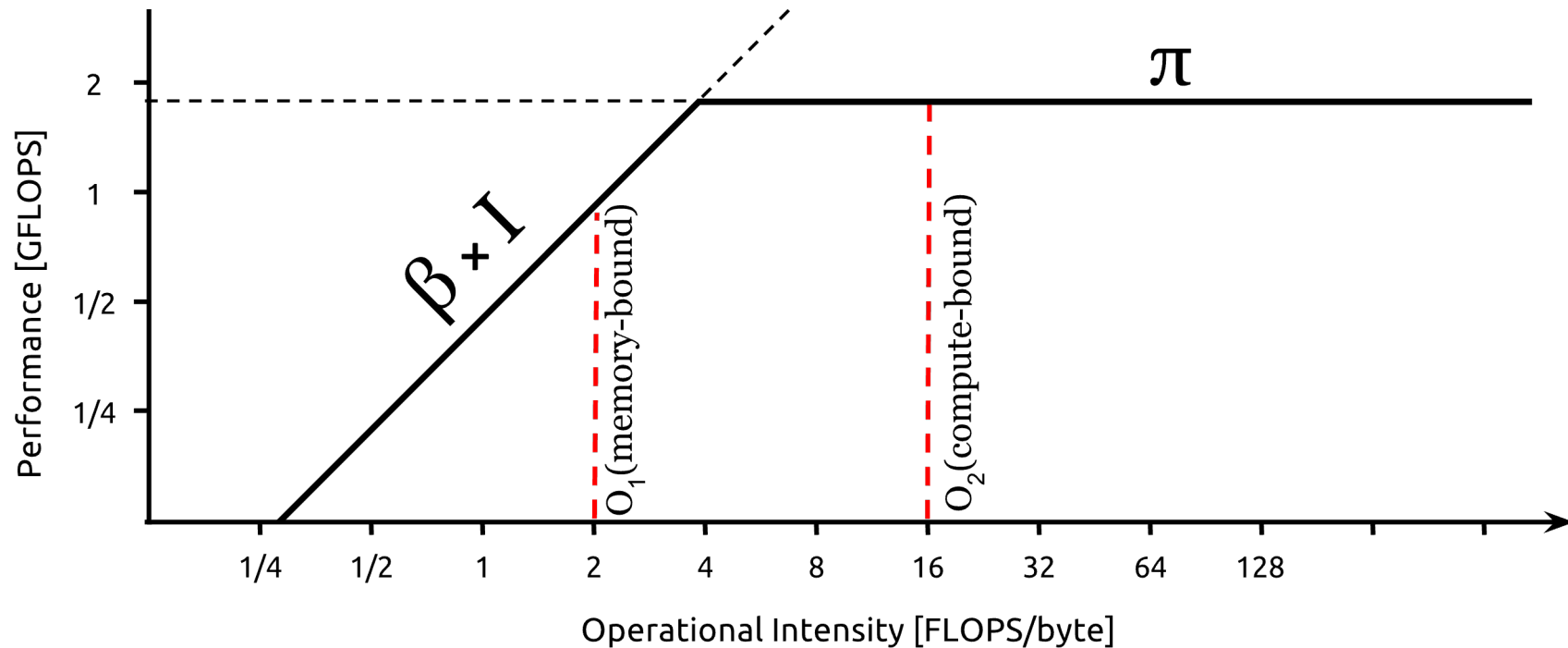
Roofline model

- Only three parameters: Peak performance, peak bandwidth vs Arithmetic intensity
 - Peak performance is derived from benchmarking FLOPs or GFLOPs
 - Peak bandwidth from manuals of the memory subsystem
 - Ridge point: where the horizontal and diagonal lines meet = minimum AI required to achieve the peak performance



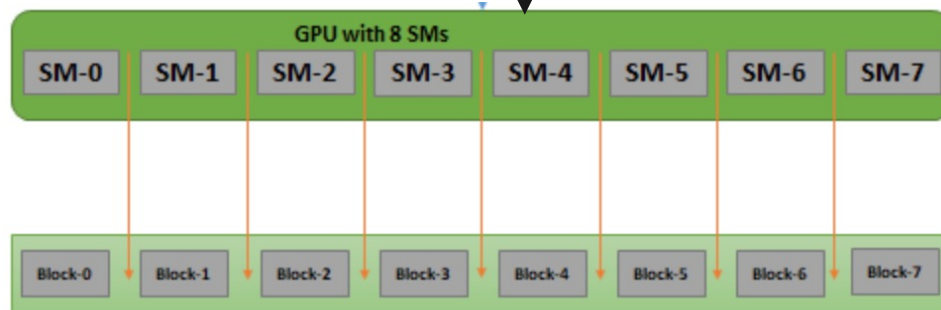
Roofline model

A given kernel or application is then characterized by a point given by its arithmetic intensity I (on the x-axis). The attainable performance P is then computed by drawing a vertical line that hits the Roofline curve.



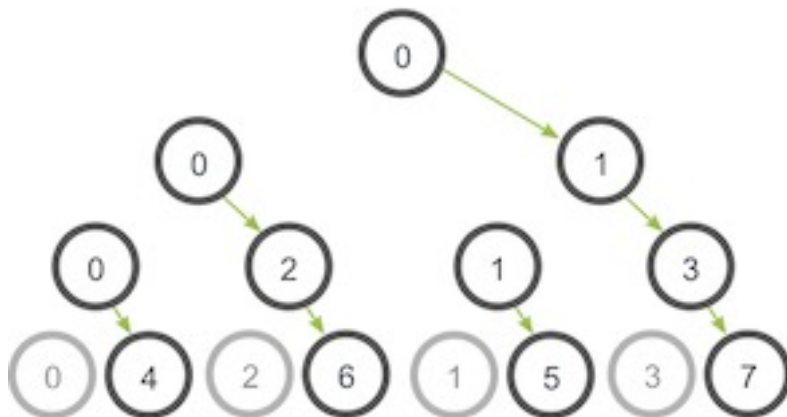
CUDA Thread Pool Abstraction

- The compiled CUDA program has eight CUDA blocks. The CUDA runtime can choose how to allocate these blocks to multiprocessors as shown with streaming multiprocessors (SMs).
- For a larger GPU with eight SMs, each SM gets one CUDA block. This enables performance scalability for applications with more powerful GPUs without any code changes.



Wieso MPI_Bcast schneller ist:
root node broadcasts and other nodes help broadcasting after receiving broadcast
tree based communication

A smarter tree-based communication in MPI_Bcast



- process zero sends data to process one.
- process zero also sends the data to process two in the second stage.
- process one helps the root process by forwarding the data to process three.
- During the second stage, two network connections are being utilized at a time.
- The network utilization doubles at every subsequent stage of the tree communication until all processes have received the data.

Limited Shared Memory

load A and B in several steps,
Calculate the intermediate
result for C from this

